

MQTT: Get Started with IoT Protocols

MQTT is a machine-to-machine, Internet of Things connectivity protocol. It is an extremely lightweight publish-subscribe communication model, useful for connections in remote locations where a small code footprint is the order of the day.

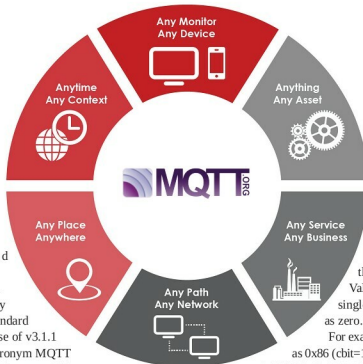
MQTT is a telemetry protocol based on the publish-subscribe communication model. It was initially developed by IBM and is of OASIS standard now, with the latest release of v3.1.1 in 2013. Formerly, the acronym MQTT stood for MQ Telemetry Transport. This lightweight, open, simple and easy-to-implement protocol is suitable for constrained environments and applicable in M2M and IoT contexts. MQTT tools have a minimal code footprint and allow optimal bandwidth usage. It runs on TCP by default, but can be run on any ordered, loss-less, bi-directional protocol.

MQTT nodes communicate in a one-to-many mapping model, where a message sent by one client (the publisher) is delivered to many clients (subscribers) through topic names. Messages are exchanged via a central node known as the broker.

The MQTT packet structure

A MQTT packet consists of a fixed header with a minimum size of 2 bytes and a maximum size of 5 bytes, which makes it very lightweight compared to other protocols like HTTP, XMPP, etc. After the fixed header, variable header elements and optional payload may follow. The first byte of the header comprises a 4 bit packet type and a few flags known as header flags. The second byte onwards represents the remaining length covering the optional header and payload, excluding the fixed header.

MQTT uses a variable length scheme to fit the remaining length with a minimum of one byte and up to four bytes encoded, as follows. The most significant bit of each byte is considered as the continuation bit, where 0 means the last



byte of length and 1 means that the length follows in the byte. The continuation bit of the fourth byte must be zero. Values up to 127 are encoded in a single byte with the continuation bit as zero.

For example, a value of 390 is encoded as 0x86 (cbit=1, LSB 7 bits=6) in byte1 and 0x03 (cbit=0, LSB 7 bits=3) in byte2, where the remaining length is computed as $[6 \times 128^4 + 6 \times 128^3]$.

If a, b, c and d are magnitudes of the least significant 7 bits of each byte in the 0-127 range, length is computed as $a \times 128^6 + b \times 128^5 + c \times 128^4 + d \times 128^3$. This allows a maximum payload up to 256MB, but one should consider TCP limits to avoid fragmentation at low levels and optimal transfers.

Control codes for various packet types are shown in Figure 3.

Let's have a quick look at the features of MQTT, some of its tools and libraries before going into the details of header flags, variable header components and payload elements.

Topics

MQTT packets sent by publishers are delivered to all subscribers registered for the same topic or matching topic filter. The topic name can be hierarchical with '/' as the delimiter in between to avoid collisions; e.g., a topic name can be in the form of a country/state/city/temperature, city/area/street/light, hospital/ward/patient/pulse, etc. Most cloud platforms suggest user-specific patterns for starting levels of a topic.

Two wild cards are allowed in topic names, where '+' matches a single level in the topic name and '#' matches multiple levels; e.g., let's consider four published topics: (1) a/b/c/d, (2) a/b/x/y, (3) a/b/c/y, and (4) a/b/x/d. Now the topic filter a/b/+/d matches (1) and (4); a/b/c/+ matches (1) and (3); and a/b/# matches all.